Self-Evolving Agent Optimization with Multi-Provider Coding Agents: A Study on AppWorld with auto-agent and Kiro CLI

Fabio Nonato de Paula
nonatofabio@github

March 2026

Abstract

We present an empirical study of **auto-agent**, a self-evolving agent optimization system that autonomously improves AI agents through iterative, hypothesis-driven code modifications. We extend auto-agent with a provider abstraction supporting multiple coding agent backends, including a new integration with Kiro CLI alongside the original Claude Code support. We evaluate the system on AppWorld, a benchmark of 732 interactive coding tasks across 9 simulated real-world applications. Using Kiro CLI as the orchestrating coding agent, we measure how effectively auto-agent improves a minimal AppWorld agent when the target agent uses six different Claude models via Amazon Bedrock: Claude 3.5 Haiku, Claude Haiku 4.5, Claude Sonnet 4, Claude Sonnet 4.5, Claude Sonnet 4.6, and Claude Opus 4.6. Starting from 0% accuracy across all configurations, auto-agent achieves up to 80% accuracy over three optimization iterations for Opus 4.6—the only model to improve monotonically across all iterations—while Sonnet-class models plateau at 20–40% after a single iteration. These results demonstrate that autonomous agent improvement is feasible and that the target model's capability is the primary determinant of both optimization ceiling and the system's ability to sustain multi-iteration gains.

1 Introduction

The development of AI agents that interact with external tools and APIs is a rapidly growing area, yet improving these agents remains largely a manual process. Developers analyze failure cases, hypothesize improvements, implement changes, and evaluate results—a cycle that is time-consuming and difficult to scale.

auto-agent[1] addresses this by automating the optimization loop. Given a target agent repository and a golden dataset of expected behaviors, auto-agent spawns a coding agent to analyze failures, implement fixes, run evaluations, and decide whether to accept or rollback changes. Each iteration produces a hypothesis—a single, scoped improvement attempt—with accumulated learnings persisted across iterations.

In this work, we make three contributions:

1. **Provider abstraction**: We decouple auto-agent from Claude Code, introducing a pluggable provider interface that supports both Claude Code and Kiro CLI as coding agent backends.

2. **Kiro CLI integration**: We implement a Kiro CLI provider that uses steering files for system prompt injection while preserving full tool access, resolving a limitation where Kiro's `-agent` flag disables built-in tools.

[1] `https://github.com/alfonsograziano/auto-agent`

3. **AppWorld evaluation**: We evaluate auto-agent's optimization capability on AppWorld, a challenging benchmark of interactive coding tasks, across four Claude model tiers serving as the target agent's LLM backbone.

2 Background

2.1 auto-agent

auto-agent uses a two-repository architecture: an *orchestrator* that controls the optimization loop, and a *target agent* repository containing the agent being improved. The orchestrator spawns a coding agent inside the target repository with full context about the job objective, prior learnings, and evaluation results.

Each optimization iteration follows a fixed protocol:

1. Read the baseline report, accumulated memory (`MEMORY.md`), and job configuration
2. Create a git branch for the new hypothesis
3. Spawn the coding agent with full context
4. The coding agent analyzes failures, implements a fix targeting one failure class, and runs evaluations
5. Parse the decision (`CONTINUE` or `ROLLBACK`) from the structured report
6. Accept or discard the hypothesis branch accordingly

A shared `MEMORY.md` file persists learnings across iterations, preventing the system from repeating failed strategies.

2.2 AppWorld

AppWorld [1] is a benchmark of 732 autonomous agent tasks built on a simulated world of 9 real-world applications (Spotify, Amazon, Venmo, etc.) with 457 APIs and approximately 100 simulated users. Tasks require interactive Python code generation to call APIs, navigate paginated results, and complete multi-step workflows. Evaluation is state-based: database unit tests verify whether the agent achieved the correct outcome regardless of the code path taken.

AppWorld is particularly challenging because agents must:

- Discover correct API signatures through documentation
- Handle authentication flows (login, access tokens)
- Navigate paginated API responses
- Compose multi-step workflows across applications

2.3 Kiro CLI

Kiro CLI[2] is a terminal-based AI coding assistant. Unlike Claude Code, which provides `-system-prompt` and `-add-dir` flags for programmatic control, Kiro CLI requires alternative integration strategies. We discovered that Kiro's `-agent` flag disables all built-in tools (file read/write, shell execution), making it unsuitable for auto-agent's use case. Instead, we use *steering files*—markdown files placed in `.kiro/steering/` with `inclusion: always` frontmatter—which inject system prompts while preserving full tool access.

[2] `https://kiro.dev/cli/`

3 System Design

3.1 Provider Abstraction

We introduce an `AgentProvider` interface with three methods:

```
interface AgentProvider {
  readonly name: string;
  assertInstalled(): void;
  run(opts: AgentRunOptions): Promise<number>;
  cleanup?(): Promise<void>;
}
```

The `ClaudeProvider` wraps Claude Code's CLI with `-system-prompt` and `-add-dir` flags. The `KiroProvider` writes a temporary steering file to `.kiro/steering/` and folds the job directory context into the user prompt. Provider selection is configured via a `**Provider**` field in the job's `JOB.md`.

3.2 AppWorld Target Agent

We construct a minimal target agent consisting of three files:

- `agent.py`: Contains the system prompt, LLM calling logic, and a `solve_task()` function implementing a ReAct-style loop
- `llm_call.py`: A subprocess-based LLM caller that invokes Amazon Bedrock outside of AppWorld's `freezegun` time patching (which otherwise breaks AWS SigV4 signature validation)
- `run_eval.py`: An evaluation runner that iterates over AppWorld tasks and outputs JSON results

The agent starts with a deliberately minimal system prompt and no API exploration logic, establishing a 0% baseline that auto-agent must improve.

4 Experimental Setup

4.1 Models

We evaluate six Claude models available via Amazon Bedrock, spanning three capability tiers:

Model	**Tier**	**Generation**
Claude 3.5 Haiku	Haiku (small)	Previous
Claude Haiku 4.5	Haiku (small)	Current
Claude Sonnet 4	Sonnet (medium)	Previous
Claude Sonnet 4.5	Sonnet (medium)	Current
Claude Sonnet 4.6	Sonnet (medium)	Latest
Claude Opus 4.6	Opus (large)	Latest

Table 1: Target agent models evaluated. All accessed via Bedrock cross-region inference profiles in `us-east-1`.

4.2 Configuration

All experiments use:

- **Orchestrating agent**: Kiro CLI (the coding agent that modifies the target agent)
- **Dataset**: 10 AppWorld `train` tasks (Spotify queries, mutations, multi-app workflows)
- **Max iterations**: 3 per model
- **Forbidden files**: `run_eval.py`, `llm_call.py`, `data/`
- **Modifiable**: `agent.py` (system prompt, agent loop logic, helper functions)

The orchestrating Kiro agent receives the same job configuration for all four experiments, ensuring that differences in outcomes are attributable to the target model's capability rather than prompt variation.

5 Results

5.1 Quantitative Results

Model	Baseline	Iter 1	Iter 2	Iter 3	Final
Claude 3.5 Haiku	0%	0% (C)	0% (C)	0% (C)	0%
Claude Haiku 4.5	0%	**10%** (C)	0% (R)	0% (R)	10%
Claude Sonnet 4	0%	**40%** (C)	40% (R)	30% (R)	40%
Claude Sonnet 4.5	0%	**40%** (C)	40% (R)	10% (R)	40%
Claude Sonnet 4.6	0%	**20%** (C)	30% (R)	30% (R)	20%
Claude Opus 4.6	0%	30% (C)	40% (C)	**80%** (C)	**80%**

Table 2: Accuracy progression across optimization iterations. (C) = CONTINUE (accepted), (R) = ROLLBACK (rejected). All start from 0% baseline. Opus 4.6 is the only model to improve monotonically across all three iterations.

Model	Total Time	Avg Iteration
Claude 3.5 Haiku	39m 24s	9m 32s
Claude Haiku 4.5	43m 32s	10m 44s
Claude Sonnet 4	49m 43s	11m 44s
Claude Sonnet 4.5	1h 4m 54s	17m 4s
Claude Sonnet 4.6	1h 3m 11s	16m 55s
Claude Opus 4.6	1h 2m 18s	15m 19s

Table 3: Wall-clock time for each experiment (baseline + 3 iterations).

5.2 Qualitative Analysis

Iteration 1 is decisive for most models. For Haiku and Sonnet-class models, the first iteration produced the only accepted improvement. Subsequent iterations either matched or regressed. However, Opus 4.6 broke this pattern entirely—it improved monotonically across all three iterations (30% → 40% → 80%), suggesting that more capable models can sustain multi-iteration optimization where smaller models plateau.

Structural improvements are model-independent. The Kiro coding agent made similar structural changes across all six experiments:

- Added an API exploration step before LLM reasoning
- Fixed the system prompt with correct API parameter names
- Added password retrieval and API documentation search

These changes were necessary but not sufficient—the target model's ability to *use* the provided information determined whether accuracy improved.

Haiku-class models hit a capability ceiling. Claude 3.5 Haiku could not successfully complete any AppWorld task even with correct API documentation in context. It consistently generated incorrect API calls, misunderstood response formats, and failed to handle pagination. Claude Haiku 4.5 showed marginal improvement (10%), successfully completing one simple lookup task.

Sonnet-class models benefit but plateau. Sonnet 4 and 4.5 both achieved 40% accuracy after a single iteration, solving difficulty-1 Spotify lookups. Sonnet 4.6 achieved only 20%, possibly due to different code generation patterns. All Sonnet models failed to improve beyond iteration 1.

Opus 4.6 demonstrates sustained optimization. The most striking result is Opus 4.6's trajectory: 0% → 30% → 40% → 80%, with all three hypotheses accepted. This suggests that Opus-class models can not only follow improved scaffolding but also benefit from the accumulated learnings in `MEMORY.md` across iterations. The coding agent was able to identify progressively more sophisticated failure classes and implement fixes that Opus could reliably execute.

MEMORY.md captures useful learnings. The accumulated memory correctly identified:

- Working patterns: "API exploration infrastructure is now in place"
- Failed approaches: "Simply providing API docs in the prompt is not enough for smaller models"
- Known blockers: "Paginated results require iterative fetching that the model doesn't reliably implement"

5.3 Rollback Behavior

The rollback mechanism worked correctly across all experiments. When iteration 2 or 3 produced regressions (e.g., Sonnet 4.5 dropping from 40% to 10% in iteration 3), the system correctly rejected the hypothesis and preserved the best-known state. This "safe by default" behavior is critical for autonomous operation.

6 Discussion

6.1 The Optimization Ceiling

Our results reveal a nuanced relationship between model capability and optimization ceiling. For Haiku and Sonnet-class models, auto-agent quickly identifies the right structural improvements but these only translate to gains when the model can follow the improved instructions. However, Opus 4.6's sustained improvement across all three iterations challenges the simpler narrative:

with a sufficiently capable target model, auto-agent's iterative learning mechanism—where each iteration builds on accumulated knowledge in `MEMORY.md`—can drive continued gains beyond the initial "low-hanging fruit."

This has practical implications: auto-agent is most valuable in two regimes: (1) unlocking latent capability in mid-tier models through better scaffolding, and (2) enabling sustained, compounding improvements in frontier models where each iteration can tackle progressively harder failure classes.

6.2 Kiro CLI as Orchestrator

Kiro CLI in `auto` mode (which routes to Sonnet-tier models based on credit cost analysis) proved effective as the orchestrating coding agent across all main experiments. It successfully read and understood the target agent codebase, analyzed evaluation failures, implemented targeted code changes, ran evaluations, wrote structured reports, and updated accumulated memory with learnings. The steering file approach for system prompt injection worked reliably.

6.3 Ablation: Orchestrator Model Capability

To isolate the effect of the orchestrating coding agent's capability, we ran an additional experiment holding the target model fixed at Claude Sonnet 4.6 while upgrading the Kiro orchestrator from `auto` (Sonnet-tier) to Claude Opus 4.6.

Target	**Orchestrator**	**Iter 1**	**Iter 2**	**Iter 3**	**Final**	**Credits**
Haiku 4.5	Auto (Sonnet)	10% (C)	0% (R)	0% (R)	10%	22.9
Haiku 4.5	Opus 4.6	0% (C)	0% (C)	0% (C)	0%	43.2
Sonnet 4.6	Auto (Sonnet)	20% (C)	30% (R)	30% (R)	20%	19.4
Sonnet 4.6	Opus 4.6	20% (C)	**30%** (C)	30% (C)	**30%**	31.8

Table 4: Ablation: effect of orchestrator capability on two target models. Upgrading the orchestrator to Opus helps Sonnet 4.6 (20% → 30%) but not Haiku 4.5 (10% → 0%), at roughly 1.6–1.9× the credit cost.

The results reveal an interaction between orchestrator and target capability. For Sonnet 4.6, the Opus orchestrator produced higher-quality changes that survived the rollback check across all three iterations, achieving 30% vs. 20%. For Haiku 4.5, however, the Opus orchestrator performed *worse*—its more sophisticated code modifications were beyond what Haiku could reliably execute, while the Sonnet orchestrator's simpler changes happened to unlock one task. This suggests that orchestrator–target capability matching matters: an overpowered orchestrator may generate changes that exceed the target model's ability to follow, yielding no benefit at higher cost.

6.4 Limitations

- **Small evaluation set**: We used only 10 train tasks due to time constraints. A larger evaluation would provide more statistical power.
- **Three iterations**: Opus 4.6 was still improving at iteration 3 (80% as target), suggesting more iterations could yield further gains.
- **Non-determinism**: LLM-based agents are inherently non-deterministic. Results may vary across runs.

7 Related Work

Autoresearch [2] automates the research process by having LLMs generate hypotheses, run experiments, and iterate. auto-agent applies a similar philosophy to agent development.

DSPy [3] optimizes LLM pipelines by tuning prompts and few-shot examples. Unlike auto-agent, DSPy operates at the prompt level rather than modifying agent code.

AppWorld [1] provides the benchmark used in our evaluation. The official leaderboard shows that even state-of-the-art models achieve modest scores on the challenge set, confirming the benchmark's difficulty.

8 Conclusion

We demonstrated that auto-agent, extended with Kiro CLI support, can autonomously improve an AppWorld agent from 0% to 80% accuracy for Opus 4.6 over three optimization iterations, and to 40% for Sonnet-class models in a single iteration. The provider abstraction enables fair comparison across coding agent backends, and the AppWorld benchmark provides a rigorous evaluation framework for interactive coding agents.

Three findings emerge from our experiments. First, the target model's capability is the primary determinant of the optimization ceiling: the same orchestrator making similar structural improvements produces 0–80% accuracy depending on the target model. Second, only frontier models sustain multi-iteration gains—Opus 4.6 was the sole model to improve monotonically across all three iterations, while every other model plateaued after iteration 1. Third, and perhaps most surprisingly, orchestrator–target capability matching matters: upgrading the orchestrator to Opus 4.6 helped Sonnet 4.6 (20% $\rightarrow$ 30%) but hurt Haiku 4.5 (10% $\rightarrow$ 0%), because a more capable orchestrator generates more sophisticated code changes that can exceed a weak target model's ability to follow. The optimal configuration is not simply "use the best orchestrator available," but rather one where the orchestrator's output complexity is calibrated to the target model's instruction-following capacity.

Future work should explore more iterations (Opus 4.6 was still improving at iteration 3), larger evaluation sets, the effect of seeding `MEMORY.md` with prior knowledge, and whether the orchestrator–target matching effect holds across other benchmarks and model families.

References

[1] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *ACL*, 2024.

[2] Andrej Karpathy. Autoresearch. `https://github.com/karpathy/autoresearch`, 2025.

[3] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *ICLR*, 2024.